



Smart Inventory & Warehouse Management System

Team ID: 1 | Section: 11 | Members: S Svara Haasini – 2520090170, B Sudharshini – 2520090227 | Guide: Dr. M Gayatri
Database Systems Engineering and Distributed Backend Development (25CS1302E) | KLH Bachupally Campus, CSE | PBL 2026-27

Abstract

- Centralized MySQL + FastAPI web system to manage products, stock, suppliers and warehouses for small and mid-sized businesses.
- Every transaction updates stock in real time and low-stock items are auto-flagged, replacing error-prone manual records.

Problem Statement & Objectives

- Problem:** Manual spreadsheets/registers cause slow search, duplicate data and no low-stock alerts.
- Objective 1:** Centralize data in a normalized (3NF) MySQL database.
- Objective 2:** Automate stock operations and reorder alerts using stored procedures and SQL triggers.
- Objective 3:** Give staff a simple, responsive dashboard and reports.
- Scope:** Multi-warehouse inventory, product/supplier management, stock operations, reporting and inventory analytics.

Existing vs Proposed System

Feature	Existing System	Proposed System
Data storage	Spreadsheets / paper registers	Centralized MySQL database
Relationships	None enforced	Primary/foreign-key constraints
Automation	No	Auto stock updates & alerts
Scalability	Low	Medium-High
Gap → Fix	Manual reports, static reorder levels	SQL views/procedures, demand-based reorder

Feasibility Study

- Technical:** Open-source stack (MySQL, FastAPI, HTML/CSS/JS) fully covered by the team's SQL, Python and web skills.
- Economic:** Zero licence cost; runs on one low-spec server or free-tier cloud hosting.
- Operational:** Simple dashboard and forms need minimal training; multi-user and automates reorder alerts.
- Schedule:** Completed (100%) within the trimester timeline.

Results & Testing

- Record retrieval significantly reduced using indexed SQL queries
- Stock operations validated using transaction-safe procedures
- Low-stock alerts verified against reorder thresholds
- PK/FK constraints, views, procedures and triggers tested successfully

Conclusion

- Replaces manual records with a fast, accurate, centralized system.
- Meets all objectives using joins, views and stored procedures.

Future Scope

- Barcode/QR scanning and role-based authorization (Admin / Manager / Staff)
- AI demand forecasting, real-time analytics dashboard
- Expiry tracking (FIFO), mobile app, cloud deployment

Tools & Technologies

- Front-End:** HTML5, CSS3, JavaScript, Bootstrap 5
- Back-End:** Python, FastAPI + REST APIs, JWT authentication
- Database:** MySQL, MySQL Workbench (joins, views, stored procedures, triggers)
- Tools:** VS Code, Uvicorn ASGI Server, MySQL Workbench

System Architecture

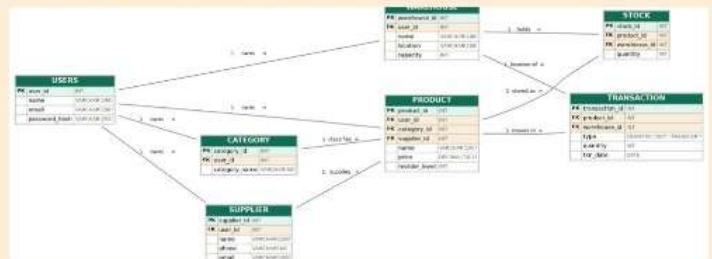
Presentation Layer HTML • CSS • JS • Bootstrap

Application Layer FastAPI: REST routes, validation, JWT

Data Access Layer Python connectors → SQL queries

Database Layer MySQL: Products, Stock, Transactions...

ER Diagram & Database Design



Implementation Screenshots



Dijkstra Route Optimizer

Dashboard overview

